Verslag labo 1

Werkwijze

We begonnen met het installeren van alle nodige software om gebruik te kunnen maken van de CUDA API. Hierna hebben we een voorbeeldprogramma opgenomen in Visual Studio Code om zo te testen of alles in orde was om te kunnen starten met het programmeren. We hebben tijdens het programmeren van onze opdracht ons vooral gebaseerd op de code die we vonden in het programma *vectorAdd.cu*, te vinden onder cuda-samples.

Code

Onze code bestaat uit 2 delen, de void flip_array en de main loop. De void flip_array is van het type `__global__` waaraan we 2 pointerarrays (`array_input` en `array_output`) meegeven en een integer die de grootte van de array voorstelt. Verder wordt hierin de output omgekeerd weergegeven door gebruik van de expressie `n-index-1` . In de main loop worden standaard variabelen aangemaakt waarmee we bepaalde bewerkingen kunnen uitvoeren. De grootte van `array_size` wordt bepaald door de verhouding van de input array op de grootte van een integer. Zo weten we hoeveel blokken er zijn. Verder maken we gebruik van de `cudaMalloc`-functie om geheugen op de GPU toe te wijzen. Hierna maken we gebruik van de `cudaMemcpy`-functie om gegevens te kopiëren van de CPU naar de GPU (en dus naar ons nieuw gealloceerde geheugen). Vervolgens delen we de grootte van de array door het aantal threads per block, en ronden we de uitkomst hiervan af naar boven. Hierdoor zorgen we ervoor dat zeker alle elementen van de array verwerkt worden. Nu hebben we alle gegevens dat we nodig hebben om de `flip_array` functie te kunnen toepassen, wat we vervolgens dus ook doen. Hierna kopiëren we terug alle resultaten met behulp van de `cudaMemcpy`-functie, maar ditmaal van de GPU naar de CPU. Ten slotte printen we de output array naar de terminal en maken we het gealloceerde geheugen op de GPU vrij.